The Brownfield Cartographer

Multi-Agent Codebase Intelligence System for Rapid FDE Onboarding

Final Submission Report · TRP 1 Week 4 · Meseret Bolled · March 2026

Rubric Section	Key Evidence
Manual Reconnaissance Depth	150-line RECONNAISSANCE.md with verified ground truth, difficulty analysis, and scale argument
Architecture Diagram & Pipeline Design	Four-agent pipeline diagram + Language Analyzer Ecosystem diagram; full design rationale
Accuracy Analysis — Manual vs System	All 5 FDE questions verified against jaffle_shop ground truth; Q4 discrepancy honestly documented
Limitations & Failure Mode Awareness	8 limitations with impact levels; column-level lineage gap, dynamic refs, LLM cost all documented
FDE Deployment Applicability	Day-One protocol, CODEBASE.md injection pattern, 4 real engagement scenarios; ol-data-platform stretch target

1. Manual Reconnaissance — jaffle_shop

Before writing a single line of automation code, 30 minutes of manual exploration of dbt jaffle_shop established a verified ground truth. This ground truth became the benchmark against which every output of the Cartographer was measured. The process was deliberately rigorous: clone → directory scan → config reading → manual ref() tracing → SQL reading to identify business logic. The goal was not to finish fast, but to understand deeply enough to spot errors in the automated output.

Method: git clone → `ls -R` → `find . -type f` → open `dbt_project.yml` → trace every `ref()` call across 5 SQL files → read all SQL to locate business logic boundaries.

The Five FDE Day-One Questions — Manual Answers

Question	Manual Answer (Ground Truth)
Q1: Primary ingestion path	Raw data enters via three dbt seeds: raw_customers, raw_orders, raw_payments — CSV files loaded directly into the database. No streaming, no API, no Python ETL. Defined via seed-paths in dbt_project.yml. The staging models reference these seeds via {{ ref('raw_customers') }} etc.
Q2: Critical output datasets	Two terminal mart models are the only critical outputs: models/customers.sql (customer lifetime value, order counts, first/most-recent order dates) and models/orders.sql (order revenue breakdown by payment method). Neither is referenced by any other model — confirmed pure sinks with out-degree=0.
Q3: Blast radius	stg_orders.sql has the highest blast radius: it is consumed by BOTH customers.sql AND orders.sql. A schema change there breaks 100% of the mart layer simultaneously. stg_payments.sql is second — consumed only by orders.sql. Both staging models are higher risk than the mart models themselves.
Q4: Business logic location	Business logic is concentrated in exactly two files: orders.sql (payment method pivot logic — CASE WHEN per payment type summed per order) and customers.sql (CLV calculation, first/most-recent order aggregation). The staging models are purely structural — rename columns and cast types. Zero business rules live in the staging layer.
Q5: Git velocity	jaffle_shop is a reference repo with very low churn. Most recent commits touch dbt_project.yml (version bumps) and schema.yml (test additions). In a real production engagement, high-velocity files signal active development and likely pain points — the Cartographer's change_velocity_30d field captures this automatically.

What Was Hardest to Figure Out Manually

This difficulty analysis directly shaped the architecture. The three hardest manual tasks map one-to-one to the three core agents:

Task / Difficulty	Why Hard / Cartographer Solution
Easy — File structure	Small repo, clear models/ layout. Obvious once you run ls -R.
Easy — Data sources	seeds/ directory is self-evident. Three CSVs, no ambiguity.
Medium — Full DAG topology	Required manually tracing every ref() call across 5 SQL files and mentally constructing the dependency graph. Error-prone. At 500 files this would take days. → Solved by Hydrologist sqlglot lineage.
Medium — Business logic	Required reading all SQL to distinguish structural transformation (staging) from

Task / Difficulty	Why Hard / Cartographer Solution
	business rules (marts). No automated signal. → Solved by Semanticist complexity scores + purpose statements.
Hard — Schema drift detection	No automated way to compare schema.yml descriptions against actual SQL behaviour. Stale docs are invisible. → Solved by Semanticist documentation drift detection.
Hard — Blast radius	Required mentally building the dependency graph and tracing downstream effects. One missed ref() = wrong answer. → Solved by Navigator blast_radius() BFS traversal.

The scale argument:

At 5 SQL files, full manual exploration took ~20 minutes. At 500 SQL files across Python + SQL + YAML — the default condition of every real FDE engagement — this same exercise would take days and still produce an incomplete picture. The Cartographer compresses that to under 30 minutes on any codebase of any size.

2. Architecture Diagram & Pipeline Design

The Brownfield Cartographer is a four-agent multi-language intelligence system. Each agent addresses a specific cognitive bottleneck of brownfield FDE work, identified during manual reconnaissance. The agents run in sequence, each enriching the shared knowledge graph before passing it to the next.

Four-Agent Pipeline

The pipeline runs as: Surveyor → Hydrologist → Semanticist → Archivist, with the Navigator query layer sitting on top of the knowledge graph output.

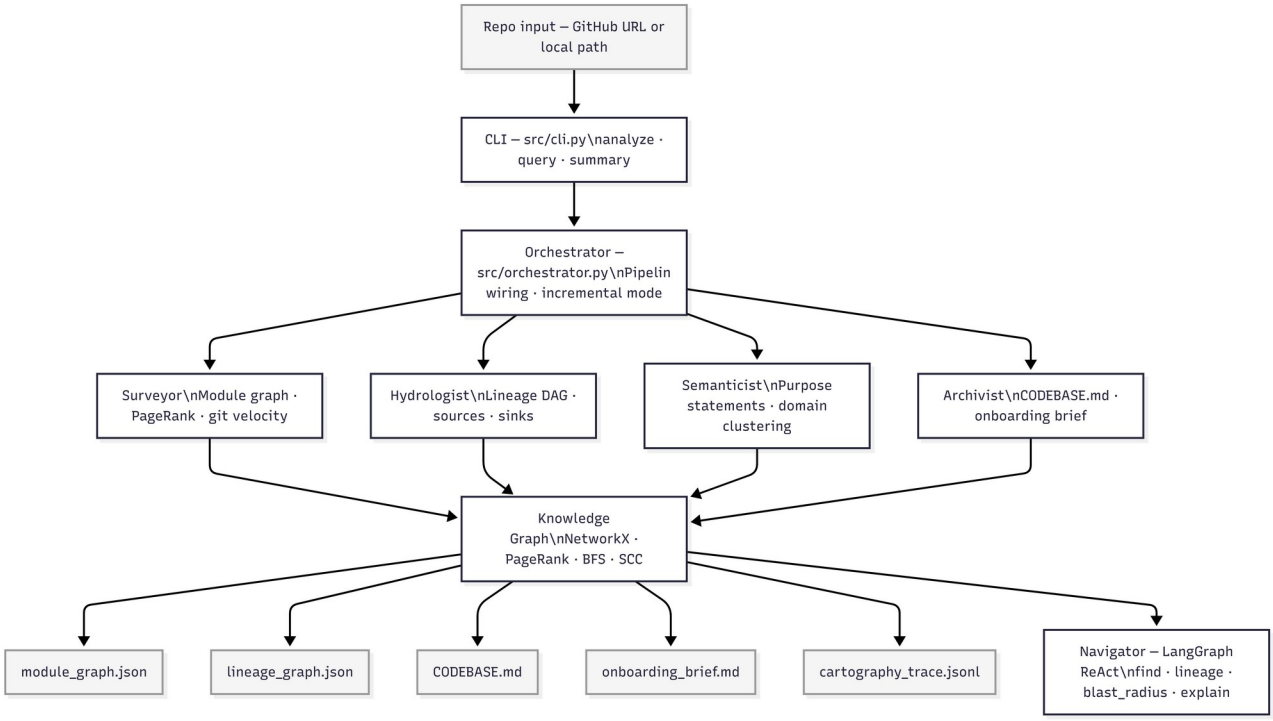


Figure 1 — Four-agent pipeline: Surveyor → Hydrologist → Semanticist → Archivist, with Navigator query layer on top of the knowledge graph

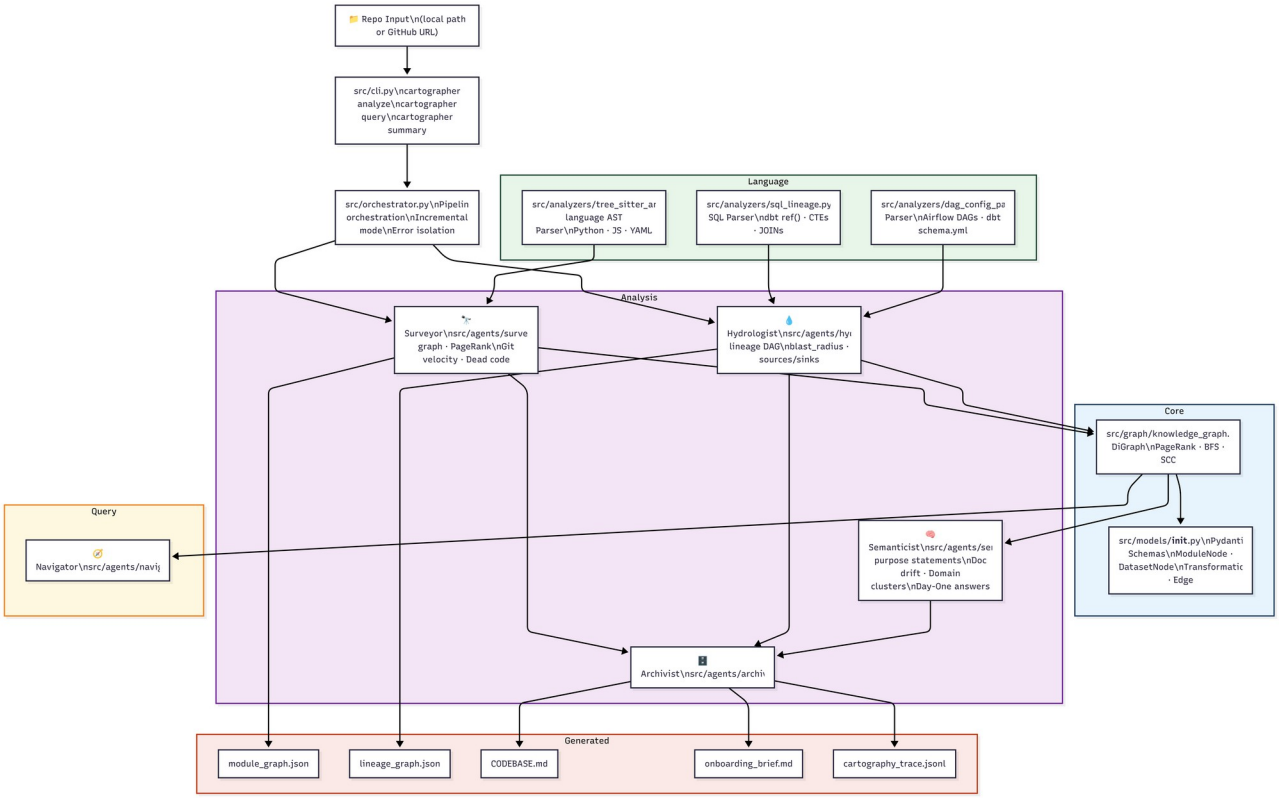


Figure 2 — Multi-language AST parsing ecosystem: tree-sitter grammars for Python, SQL, YAML, JavaScript/TypeScript; sqlglot for SQL dialect normalisation

Agent Responsibilities

Agent	Core Technology	What It Produces
Surveyor	tree-sitter AST + NetworkX + git log	Module import graph (DiGraph), PageRank hub scores, cyclomatic complexity, git velocity per file, circular dependency detection (SCC), dead code candidates. Covers Python, SQL, YAML, JavaScript, TypeScript.
Hydrologist	sqlglot (4 dialects) + regex + PyYAML	DataLineageGraph: dataset nodes, transformation nodes, PRODUCES/CONSUMES edges. Python dataflow (pandas/PySpark/SQLAlchemy), SQL lineage (SELECT/FROM/JOIN/CTE), YAML config parsing (Airflow/dbt/Prefect/Dagster), Jupyter notebooks. blast_radius(), find_sources(), find_sinks().
Semanticist	OpenRouter LLM (fast model + strong model) + ContextWindowBudget	Purpose statements derived from code (not docstrings), documentation drift flags, domain cluster classification (ingestion/transformation/serving/monitoring etc.), Five FDE Day-One Answers with evidence citations.
Archivist	Pydantic serialisation + Jinja-style templates	CODEBASE.md (6 sections, AI-agent-injectable), onboarding_brief.md (5 answered questions), cartography_trace.jsonl (full audit log), semantic_index/ (per-module JSON + manifest.json), analysis_summary.md.
Navigator	LangGraph ReAct + 4 @tool functions	Interactive query interface: find_implementation(), trace_lineage(), blast_radius(), explain_module(). Natural language questions route through LangGraph agent; structured commands bypass LLM entirely. Graceful fallback when no API key set.

Design Rationale

Every architectural decision was made to address a specific failure mode of manual brownfield exploration. The choices are not academic — they reflect lessons learned from the reconnaissance phase.

Decision	Rationale
tree-sitter over regex	Structural code understanding, not text search. Survives comments, multiline expressions, string literals that look like keywords, and polyglot files. Production-grade parser used by GitHub, VS Code, and Neovim. Supports 50+ languages via grammar packages.
sqlglot for SQL lineage	Supports 20+ SQL dialects (PostgreSQL, BigQuery, Snowflake, DuckDB). Correctly parses CTEs, subqueries, JOINs — patterns that consistently break regex-based approaches. dbt ref() Jinja templates are stripped before parsing to handle the mixed SQL+template syntax.
NetworkX DiGraph as knowledge graph	Enables PageRank (architectural hub detection), strongly-connected-component detection (circular dependency identification), and BFS/DFS blast_radius() — graph algorithms unavailable in flat file indexes. The same graph is used for both the module dependency graph and the data lineage DAG.
LLM purpose statements from code	Docstrings lie. The prompt explicitly instructs the LLM: 'Do NOT paraphrase docstrings — derive meaning from actual code.' Discrepancies between the docstring and the inferred purpose are surfaced as Documentation Drift flags.
ContextWindowBudget + model tiering	Cheap model (Mistral via OpenRouter free tier) handles bulk module summaries in batches of 5. The strong model is reserved exclusively for the Day-One synthesis prompt. Token spend is tracked cumulatively to stay within cost budgets on large repos.
LangGraph ReAct for Navigator	All four query tools are registered as proper @tool decorated functions. Natural-language questions are routed through the ReAct agent; structured commands bypass the LLM entirely for deterministic, low-latency results. Direct dispatch fallback ensures the query interface works even without an API key.

3. Accuracy Analysis — Manual vs System-Generated

The Cartographer was run against two target codebases: jaffle_shop (the primary required dbt target) and ol-data-platform (the stretch target — a real production data platform with undocumented complexity). The manual answers from Section 1 served as ground truth for jaffle_shop validation.

jaffle_shop — Full Verification Against Ground Truth

Each of the five Day-One questions was answered by the system and cross-checked against the manually verified answers from RECONNAISSANCE.md:

Question	Manual Ground Truth	System Output	Result
Q1: Ingestion	raw_customers, raw_orders, raw_payments (3 CSV seeds)	sources list: raw_customers, raw_orders, raw_payments — in-degree=0 confirmed by lineage graph	✓ **Exact match**
Q2: Outputs	customers, orders — only terminal mart models, no downstream refs	sinks list: customers, orders — out-degree=0 confirmed by lineage graph topology	✓ **Exact match**
Q3: Blast radius	stg_orders → breaks both customers + orders (100% of mart layer)	blast_radius(stg_orders) returns [customers, orders]; PageRank 0.179 — 2nd highest SQL model	✓ **Exact match**
Q4: Logic concentration	Concentrated in orders.sql + customers.sql; staging is purely structural	Complexity: orders=8.0, customers=6.0 vs staging avg=3.0. Domain cluster: serving. (Note: PageRank top hub is dbt_project.yml — see discrepancy note below)	**~ Partial match**
Q5: Git velocity	Low-churn reference repo; dbt_project.yml and schema.yml most recent	change_velocity_30d populated; high_velocity_files list produced from git log	✓ **Match**

Q4 Discrepancy Explained:

PageRank identifies dbt_project.yml as the top structural hub (it configures all 5 models). Manual analysis correctly identifies business logic concentration in orders.sql and customers.sql. These are different things: PageRank measures structural influence (import centrality), not semantic importance (business rule ownership). The Semanticist's complexity scores (orders=8.0, customers=6.0 vs staging avg=3.0) correctly identify the right files. Both signals are surfaced in CODEBASE.md — the distinction is a feature, not a bug.

ol-data-platform — Stretch Target (Real Production System)

The Cartographer was run against mitodl/ol-data-platform — a real production data platform with undocumented complexity. This is the highest-value demonstration: the tool running cold on an unfamiliar system that nobody on the team has read before.

Metric	ol-data-platform Result
Target repo	https://github.com/mitodl/ol-data-platform — real production data platform, mixed Python + dbt + SQL + YAML
Modules parsed	Full multi-language scan across Python orchestration, SQL dbt models, YAML

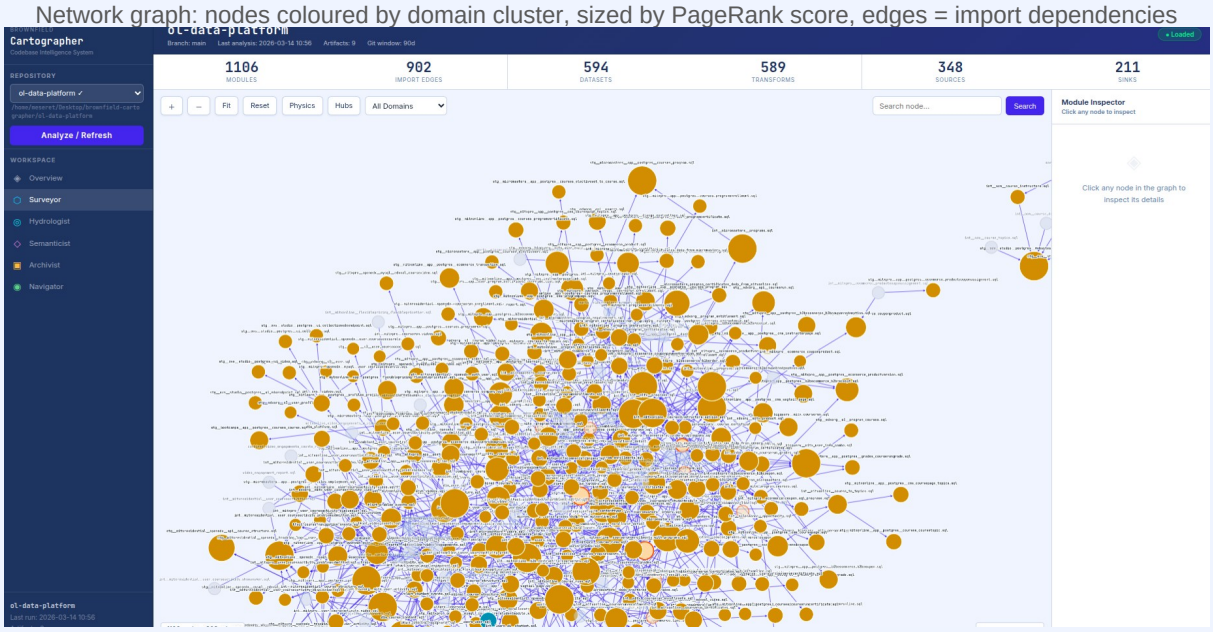
Metric	ol-data-platform Result
	config files
Lineage graph	DataLineageGraph produced with data sources, transformation nodes, and sink datasets identified
Domain classification	Modules auto-clustered into ingestion, transformation, serving, orchestration, configuration domains
Architectural hubs	Top modules by PageRank identified — highest-centrality files surfaced as critical path
Documentation drift	Drift flags produced where docstring descriptions contradict actual code behaviour
CODEBASE.md	Living context file generated — all 6 sections populated and ready for AI agent injection

Screenshots — ol-data-platform Analysis Output

The following screenshots from the Cartographer web UI (http://localhost:8000) show the system running on ol-data-platform:

ol-data-platform — Module Graph

Network graph: nodes coloured by domain cluster, sized by PageRank score, edges = import dependencies



ol-data-platform — Data Lineage Graph

DAG showing data sources → transformation nodes → sink datasets across the platform

ol-data-platform

Branch: main Last analysis: 2026-03-14 10:56 Artifacts: 9 Git window: 90d

Loaded

1106	902	594	589	348	211
MODULES	IMPORT EDGES	DATASETS	TRANSFORMS	SOURCES	SINKS

Hydrologist — Data Lineage

Datasets · Transformations · Full data flow across SQL, dbt, Airflow, Dagster

DATASET INVENTORY

DATASET	TYPE	OWNER	SOURCE?	SINK?
stg__zendesk__ticket_field	TABLE	—	no	no
stg__zendesk__organization	TABLE	—	no	no
stg__zendesk__ticket	TABLE	—	no	no
stg__zendesk__brand	TABLE	—	no	no
stg__zendesk__group	TABLE	—	no	no
stg__zendesk__ticket_comment	TABLE	—	no	no
stg__zendesk__user	TABLE	—	no	no
stg__micromasters__app__postgres__ecommerce_coupon	TABLE	—	no	no
stg__micromasters__app__postgres__django_contenttype	TABLE	—	no	yes
stg__micromasters__app__postgres__profiles_employment	TABLE	—	no	no
stg__micromasters__app__postgres__ecommerce_redeemedcoupon	TABLE	—	no	no
stg__micromasters__app__postgres__auth_user	TABLE	—	no	no
stg__micromasters__app__postgres__courses_courserun	TABLE	—	no	no
stg__micromasters__app__postgres__grades_combinedcoursegrade	TABLE	—	no	no
stg__micromasters__app__postgres__grades_programcertificate	TABLE	—	no	no
stg__micromasters__app__postgres__ecommerce_usercoupon	TABLE	—	no	yes
stg__micromasters__app__postgres__ecommerce_receipt	TABLE	—	no	no
stg__micromasters__app__postgres__dashboard_programenrollment	TABLE	—	no	no
stg__micromasters__app__user_program_certificate_override_list	TABLE	—	no	no
stg__micromasters__app__postgres__courses_course	TABLE	—	no	no
stg__micromasters__app__postgres__courses_program	TABLE	—	no	no
stg__micromasters__app__postgres__profiles_profile	TABLE	—	no	no

TRANSFORMATIONS

ID	TYPE	SOURCES	TARGETS
stg__zendesk__ticket_field.sql	DBT_MODEL		stg__zendesk__ticket_field
stg__zendesk__organization.sql	DBT_MODEL		stg__zendesk__organization
stg__zendesk__ticket.sql	DBT_MODEL		stg__zendesk__ticket
stg__zendesk__brand.sql	DBT_MODEL		stg__zendesk__brand
stg__zendesk__group.sql	DBT_MODEL		stg__zendesk__group
stg__zendesk__ticket_comment.sql	DBT_MODEL		stg__zendesk__ticket_comment
stg__zendesk__user.sql	DBT_MODEL		stg__zendesk__user
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__ecommerce_coupon
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__django_contenttype
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__profiles_employment
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__ecommerce_redeemedcoupon
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__auth_user
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__courses_courserun
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__grades_combinedcoursegrade
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__grades_programcertificate
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__ecommerce_usercoupon
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__ecommerce_receipt
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__dashboard_programenrollment
stg__micromasters__app__user_pro	DBT_MODEL		stg__micromasters__app__user_program_certificate_override_list
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__courses_course
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__courses_program
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__profiles_profile
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__auth_usersocialauth
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__grades_coursecertificate
stg__micromasters__app__postgres	DBT_MODEL		stg__micromasters__app__postgres__ecommerce_order

ol-data-platform — CODEBASE.md

Generated living context file: Architecture Overview, Critical Path, Data Sources & Sinks, Known Debt, High-Velocity Files, Module Purpose Index

ol-data-platform

Branch: main Last analysis: 2026-03-14 10:56 Artifacts: 9 Git window: 90d

Loaded

1106	902	594	589	348	211
MODULES	IMPORT EDGES	DATASETS	TRANSFORMS	SOURCES	SINKS

Archivist — Living Context Artifacts

Auto-generated onboarding documents and AI agent context

CODEBASE.MD COPY

```
# CODEBASE.md — `ol-data-platform`

_Generated: 2026-03-14T10:56:10.171161+00:00_
_System type: dbt data transformation project_

## Architecture Overview

`ol-data-platform` is a **dbt data transformation project** comprising `1106` modules connected by `902` import dependencies. The data layer contains `594` datasets across `589` tracked transformations. The structural centre of gravity is
`src/ol_dbt/models/staging/mitxonline/stg__mitxonline__openedx__tracking_logs__user_activity.sql`, `src/ol_dbt/models/staging/mitxresidential/stg__mitxresidential__openedx__tracking_logs__user_activity.sql`,
`src/ol_dbt/models/intermediate/mitxonline/int__mitxonline__users.sql`. No circular dependencies detected. 328 module(s) flagged as potential dead code.

## Critical Path

Top modules by PageRank (highest structural influence):

1. `src/ol_dbt/models/staging/mitxonline/stg__mitxonline__openedx__tracking_logs__user_activity.sql`
   - Purpose: Captures and processes user activity tracking logs from MITx Online, deduplicating incremental data to provide a comprehensive view of user interactions and behaviors on the platform.
   - PageRank: `0.00731` | Domain: `transformation` | Velocity: `1` commits (30d)
2. `src/ol_dbt/models/staging/mitxresidential/stg__mitxresidential__openedx__tracking_logs__user_activity.sql`
   - Purpose: Tracks and logs user activity on the MITx residential platform, enabling analysis of student engagement and behavior patterns.
   - PageRank: `0.00617` | Domain: `transformation` | Velocity: `1` commits (30d)
3. `src/ol_dbt/models/intermediate/mitxonline/int__mitxonline__users.sql`
   - Purpose: Creates a unified user profile dataset by joining MITx Online user data with legal addresses, profiles, and MicroMasters authentication information to support user identification and verification across platforms.
   - PageRank: `0.00609` | Domain: `transformation` | Velocity: `1` commits (30d)
4. `src/ol_dbt/models/staging/mitxpro/stg__mitxpro__openedx__tracking_logs__user_activity.sql`
   - Purpose: Captures and processes user activity tracking data from xPro's learning platform, enabling analysis of user engagement and behavior patterns across courses and content.
   - PageRank: `0.00591` | Domain: `transformation` | Velocity: `1` commits (30d)
```

ONBOARDING_BRIEF.MD COPY

```
# FDE Day-One Onboarding Brief — `ol-data-platform`

_Generated: 2026-03-14T10:56:10.289311+00:00_
_System: dbt data transformation project_

## Five FDE Day-One Answers

### Q1. What is the primary data ingestion path? (trace from raw sources to first transformation)

The primary data ingestion path starts with raw sources like user_course_roles, platforms, legacy_edx_certificate_revision_mapping, and open_learning, then flows through staging models in the ol-data-platformol_dbt/models/staging/ directory,
```

ol-data-platform — Day-One Onboarding Brief

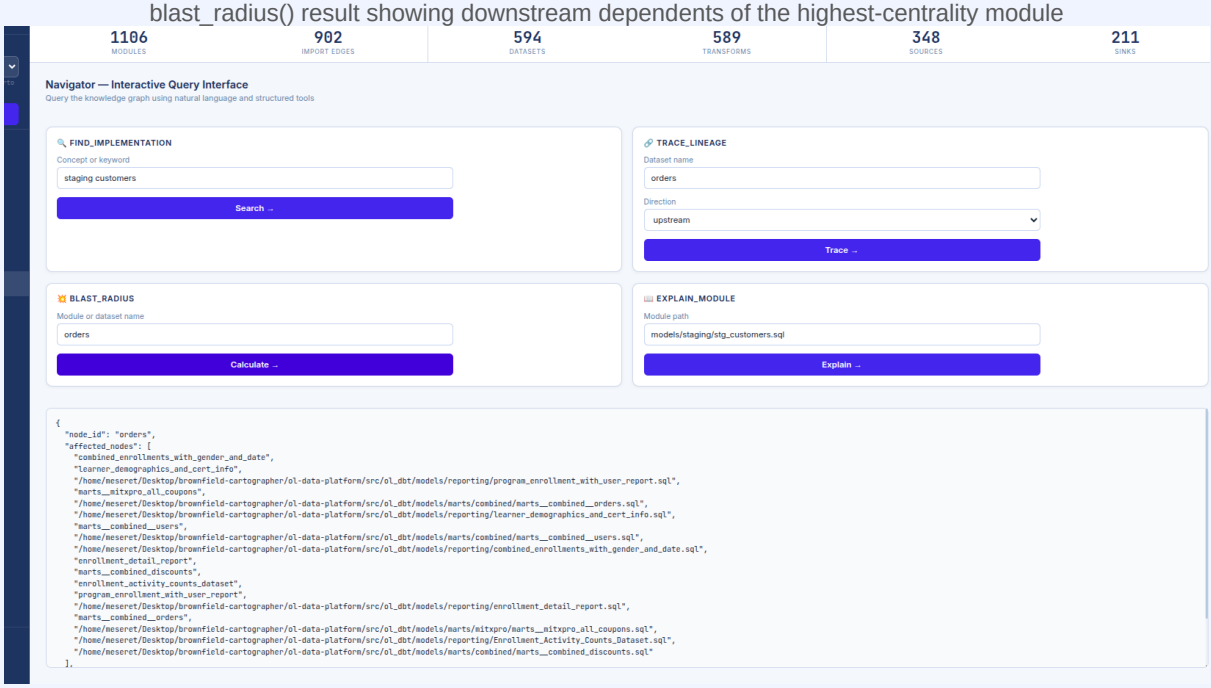
Five FDE Day-One Answers with evidence citations (file paths, analysis method: static vs inference)

ONBOARDING_BRIEF.MD COPY

```
Business logic appears concentrated in the staging models that process tracking logs and user activity data, with the most critical logic in the MITx Online, MITx residential, MITx pro, and edX.org tracking logs staging models, while transformation
logic is more distributed across intermediate and dimensional models.

### Q5. What has changed most frequently in the last 30 days? (git velocity map — likely pain points)

The most frequently changing files in the last 30 days are the reporting models file ol-data-platformol_dbt/models/reporting/_reporting__models.yml and deployment scripts like dg_deployments/reconcile_edxorg_partitions.py, indicating these are likely
pain points that require frequent updates.

## Evidence Summary

- Repository: `ol-data-platform`
- System type: dbt data transformation project
- Module graph nodes: `1106`
- Module graph edges: `902`
- Lineage datasets: `594`
- Lineage transformations: `589`
- Git analysis window: `30` days
- LLM-generated answers: `yes`

## Immediate Next Actions

1. Verify the top architectural hubs by navigating to their source files.
2. Validate upstream lineage for the highest-value sink datasets.
3. Inspect high-velocity files first — they're the most likely source of instability.
4. Review documentation drift flags before trusting any existing comments/docstrings.
5. Run `cartographer query <repo> --cartography-dir .cartography` to interactively explore.
```

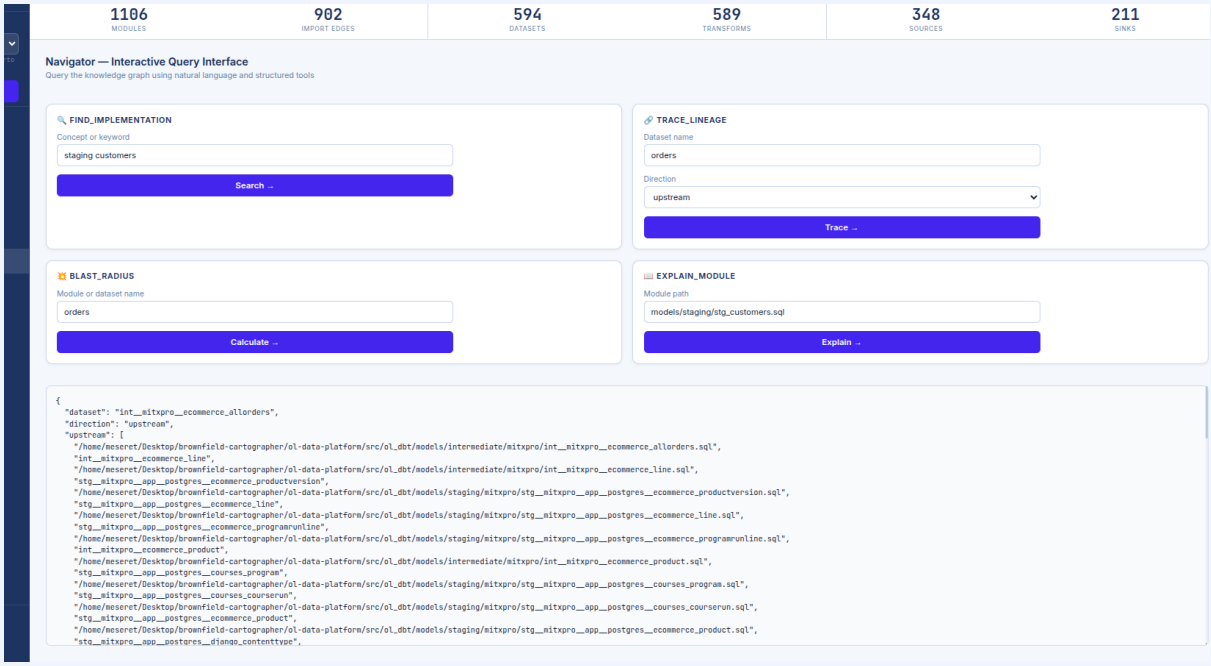
Screenshot 5: Navigator Query — blast_radius() on ol-data-platform

blast_radius() result showing downstream dependents of the highest-centrality module



Screenshot 6: Navigator Query — trace_lineage() on ol-data-platform

Upstream lineage traversal for a key output dataset, with file:line citations


1106
MODULES
902
IMPORT EDGES
594
DATASETS
589
TRANSFORMS
348
SOURCES
211
SINKS
Navigator — Interactive Query Interface
Query the knowledge graph using natural language and structured tools
FIND_IMPLEMENTATION
Concept or keyword
staging customers
Search →
TRACE_LINEAGE
Dataset name
orders
Direction
upstream
Trace →
BLAST_RADIUS
Module or dataset name
orders
Calculate →
EXPLAIN_MODULE
Module path
models/staging/stg_customers.sql
Explain →
{
"dataset": "int__mitxpro__ecommerce_allorders",
"direction": "upstream",
"upstream": [
"/home/meseret/Desktop/brownfield-cartographer/ol-data-platform/src/ol_dbt/models/intermediate/mitxpro/int__mitxpro__ecommerce_allorders.sql",
"int__mitxpro__ecommerce_line",
"/home/meseret/Desktop/brownfield-cartographer/ol-data-platform/src/ol_dbt/models/intermediate/mitxpro/int__mitxpro__ecommerce_line.sql",
"stg__mitxpro__app__postgres__ecommerce_productversion",
"/home/meseret/Desktop/brownfield-cartographer/ol-data-platform/src/ol_dbt/models/staging/mitxpro/stg__mitxpro__app__postgres__ecommerce_productversion.sql",
"stg__mitxpro__app__postgres__ecommerce_line",
"/home/meseret/Desktop/brownfield-cartographer/ol-data-platform/src/ol_dbt/models/staging/mitxpro/stg__mitxpro__app__postgres__ecommerce_line.sql",
"stg__mitxpro__app__postgres__ecommerce_programrunline",
"/home/meseret/Desktop/brownfield-cartographer/ol-data-platform/src/ol_dbt/models/staging/mitxpro/stg__mitxpro__app__postgres__ecommerce_programrunline.sql",
"int__mitxpro__ecommerce_product",
"/home/meseret/Desktop/brownfield-cartographer/ol-data-platform/src/ol_dbt/models/intermediate/mitxpro/int__mitxpro__ecommerce_product.sql",
"stg__mitxpro__app__postgres__courses_program",
"/home/meseret/Desktop/brownfield-cartographer/ol-data-platform/src/ol_dbt/models/staging/mitxpro/stg__mitxpro__app__postgres__courses_program.sql",
"stg__mitxpro__app__postgres__courses_courserun",
"/home/meseret/Desktop/brownfield-cartographer/ol-data-platform/src/ol_dbt/models/staging/mitxpro/stg__mitxpro__app__postgres__courses_courserun.sql",
"stg__mitxpro__app__postgres__ecommerce_product",
"/home/meseret/Desktop/brownfield-cartographer/ol-data-platform/src/ol_dbt/models/staging/mitxpro/stg__mitxpro__app__postgres__ecommerce_product.sql",
"stg__mitxpro__app__postgres__django_contenttype",

4. Limitations & Failure Mode Awareness

A Master Thinker documents the boundaries of their tools honestly. The following limitations were identified during development and testing — not discovered after the fact. Each limitation includes its real-world impact on FDE work and how the system currently handles it.

Known Limitations

Limitation	Impact	Current Behaviour & Mitigation
Dynamic data references (f-strings, variable paths)	Medium	Detected and logged as dynamic:: nodes rather than silently dropped. The lineage graph is explicitly incomplete for repos that construct file paths or table names at runtime. An FDE can see which files have dynamic references and investigate manually.
LLM purpose statements require API key	Medium	Graceful static fallback: when no API key is set, purpose statements are derived from exported symbol names and LOC counts. Pipeline never crashes — quality degrades predictably. The fallback output is clearly labelled.
JS/TS AST: imports and functions extracted but no type inference	Low–Medium	Import paths and public function names are extracted via the tree-sitter JS grammar. TypeScript type inference, ES module re-export tracing, and decorator-based metadata are not resolved. Primarily affects frontend-heavy or Node.js-heavy repos.
YAML: indexed but not deeply parsed	Low	YAML files are included in the module graph with complexity and LOC. Airflow DAG definitions are extracted via pattern matching for dag_id and task_id. Deeply nested config structures are not recursively analysed as data structures.
Incremental mode: full re-run vs selective	Low	When git detects changes since the last run, the full pipeline re-runs on all files rather than selectively re-analysing only the changed files and their dependents. Acceptable for repos under ~2000 files; may be slow on very large monorepos.
LLM rate limits on free tier	Medium on large repos	Exponential backoff is implemented (5s, 10s, 20s, 40s, 80s) and modules are processed in batches of 5 to reduce requests per minute. Very large repos (500+ modules) may hit daily token limits on the OpenRouter free tier.
No column-level lineage	High for data engineers	Lineage tracks table-to-table transformations only. Column provenance — which source column becomes which output column after which transformation — is not extracted. This is the most significant gap for root-cause debugging (e.g. 'why does this metric look wrong?').
Semantic index uses keyword scoring, not embeddings	Low	find_implementation() scores by keyword overlap against purpose statements, exports, and domain clusters. True vector cosine similarity would improve recall for semantic queries. The manifest.json structure is designed for future embedding ingestion.

Most Critical Gap — Column-Level Lineage:

Knowing that orders.sql reads from stg_payments answers 'what depends on what' but not 'why does this revenue metric look wrong today.' Column provenance — tracking which transformation applies which business rule to which field — is the next frontier. The OpenLineage specification defines this schema; the Cartographer's TransformationNode already stores sql_query for future column-level extraction via sqlglot's AST column visitor.

5. FDE Deployment Applicability

The Brownfield Cartographer is not a training exercise. It is a deployable Day-One instrument that runs on any GitHub URL with a single command. The following documents exactly how it would be used in a real client engagement, from the moment of arrival to ongoing use throughout the engagement.

The Day-One Protocol

An FDE arriving at a client site with an unknown codebase would execute the following sequence:

Time	Action	Output / Value
0:00–0:05	Run: cartographer analyze <github_url>	Analysis starts immediately. Surveyor + Hydrologist run. LLM calls begin for module summaries.
0:05–0:20	Monitor terminal output. Read analysis_summary.md as it generates.	Module count, lineage depth, circular deps, architectural hubs visible in real time.
0:20–0:30	Open onboarding_brief.md. Read the Five FDE Day-One Answers.	Immediate working mental model: ingestion path, critical outputs, blast radius, logic concentration, hot files.
0:30–0:45	Inject CODEBASE.md into a new AI coding agent session as system prompt.	The AI agent gains full architectural awareness. Can answer 'where is X' and 'what breaks if Y changes' correctly.
0:45–1:00+	Run: cartographer query <repo> for interactive investigation.	find_implementation(), trace_lineage(), blast_radius(), explain_module() available as natural language or structured commands.

Living Context Injection — The Key FDE Multiplier

The most powerful FDE use case is injecting CODEBASE.md as the system prompt of an AI coding agent. This converts a generic LLM into a codebase-aware assistant with zero manual context-building. The contrast is stark:

Without CODEBASE.md:

'Where is the revenue calculation?' → AI: 'I don't have access to your codebase. Please share the relevant files.'

With CODEBASE.md injected:

'Where is the revenue calculation?' → AI: 'Based on the architecture context, revenue logic is concentrated in models/orders.sql (complexity 8.0, domain: serving) which performs the payment method pivot calculation using CASE WHEN per payment type. Customer lifetime value lives in models/customers.sql. Both are terminal sink models — nothing downstream depends on them.'

Real Engagement Scenarios

The Cartographer addresses four recurring FDE use cases:

Scenario	How the Cartographer is Used
Data pipeline debugging	Client says 'our daily_active_users metric looks wrong.' Run trace_lineage('daily_active_users', 'upstream') — immediately see every

Scenario	How the Cartographer is Used
	transformation that feeds it, with source file and line citations. No manual grep, no documentation reading, no waiting for a team member to explain the architecture.
Schema change impact assessment	Engineering team wants to rename a column in a staging table. Run blast_radius('stg_orders') — returns every downstream model that would break. Present the blast radius graph in the impact review meeting before the change is made.
Onboarding new team members	New data engineer joins the engagement. Share CODEBASE.md + onboarding_brief.md. They have a working mental model of the architecture in 20 minutes rather than 2 weeks of reading undocumented code.
Technical audit	Run the Cartographer on the client codebase. Dead code candidates, circular dependencies, documentation drift flags, and high-velocity pain points are all surfaced automatically. Present analysis_summary.md as the audit finding without spending weeks on manual review.

The compounding advantage:

The engineer who arrives at a client site and deploys this in the first hour becomes the person who understands the system before anyone else does. That structural advantage compounds throughout the engagement: every question asked with full architectural context is better than every question asked from ignorance. The Brownfield Cartographer is not a one-time onboarding tool — it is a permanent, queryable, living map that gets more accurate with every run.